



Computer Vision for Autonomous Vehicles

Pose Graph Optimization

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2025



YOLO



Redmond et al, "You Only Look Once: Unified, Real-Time Object Detection", CVPR'16.

<https://www.youtube.com/watch?v=uG2UOaslx2I>

Learning-based Object Detection: YOLO

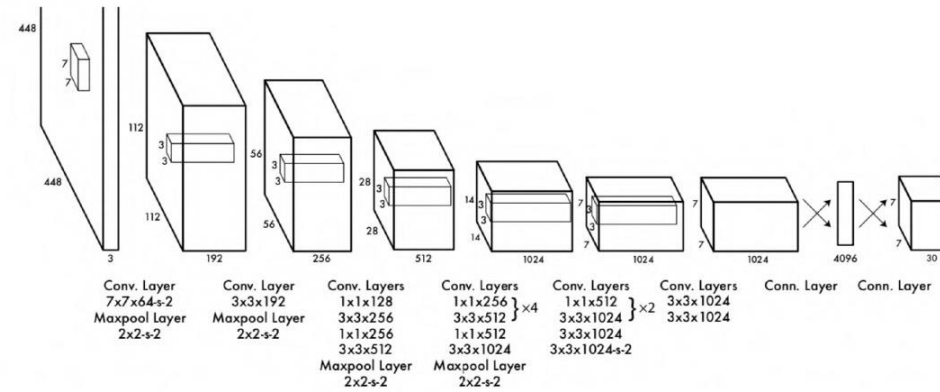


Figure 3: The Architecture. Our detection network has 24 convolutional layers followed by 2 fully connected layers. Alternating 1×1 convolutional layers reduce the features space from preceding layers. We pretrain the convolutional layers on the ImageNet classification task at half the resolution (224×224 input image) and then double the resolution for detection.

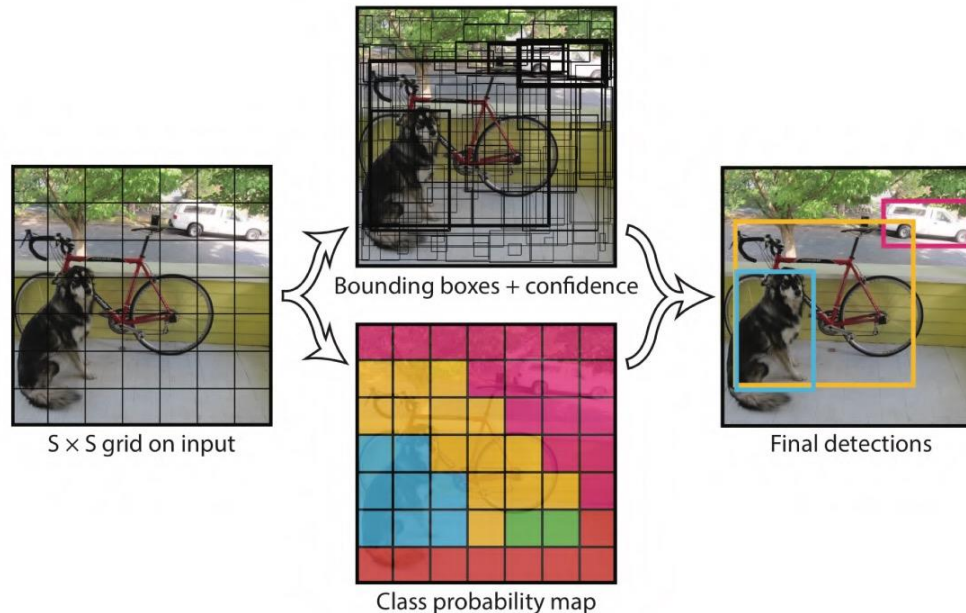


Image is split in $S \times S$ grid.

Yolo is trained to predict:

- B bounding boxes in each grid cell (x, y, h, w, confidence)
- A class label for each cell

Learning-based Object Detection: YOLO

mAP:

mean Average Precision (average precision value for recall value over 0 to 1).

- Limitations of YOLO:

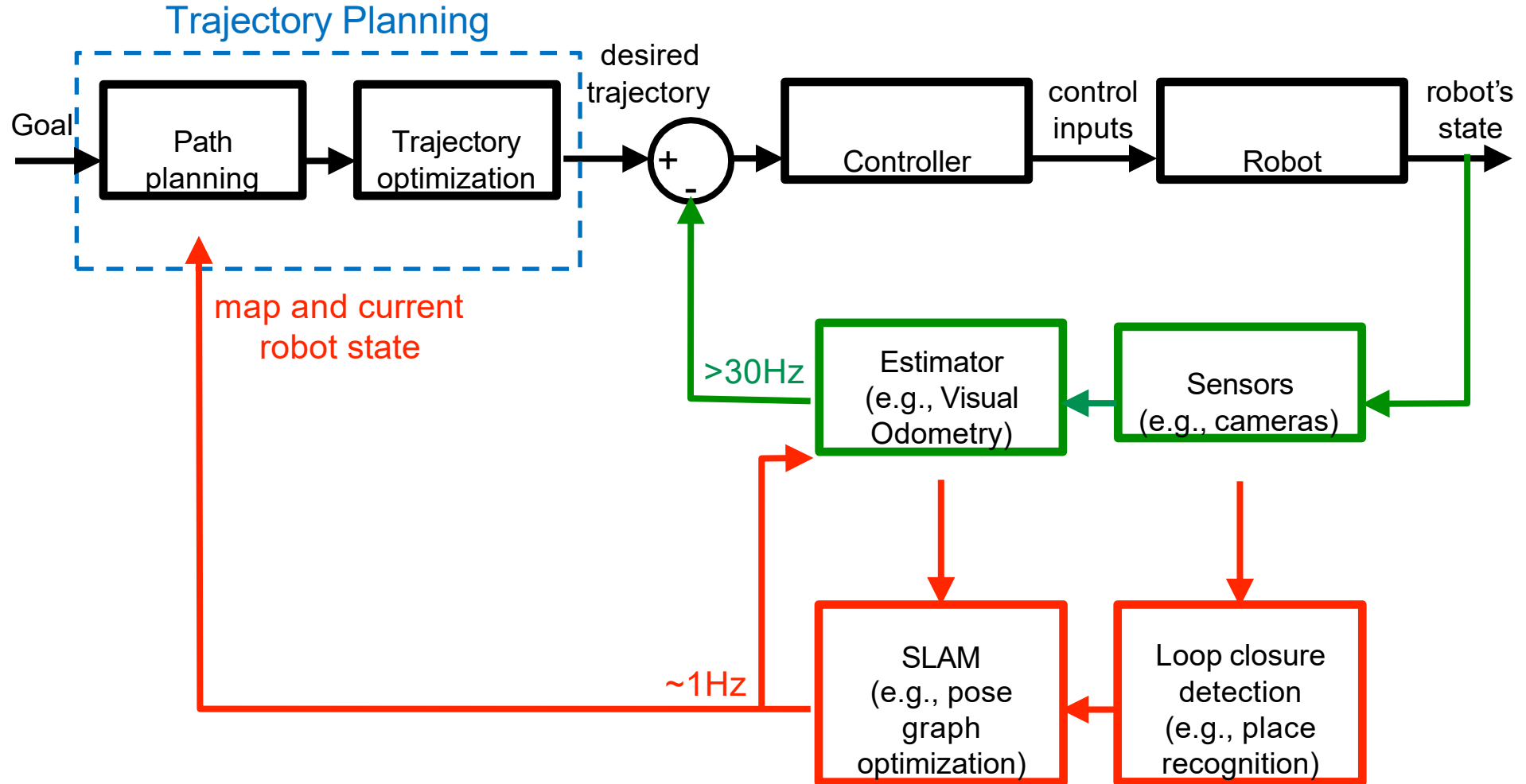
- small objects: “each grid cell only predicts B boxes and can only have one class. This spatial constraint limits the number of nearby objects that our model can predict. Our model struggles with small objects that appear in groups, such as flocks of birds.”

- generalization: fails to detect objects in new or unusual aspect ratios or configurations.

Real-Time Detectors	Train	mAP	FPS
100Hz DPM [31]	2007	16.0	100
30Hz DPM [31]	2007	26.1	30
Fast YOLO	2007+2012	52.7	155
YOLO	2007+2012	63.4	45
Less Than Real-Time			
Fastest DPM [38]	2007	30.4	15
R-CNN Minus R [20]	2007	53.5	6
Fast R-CNN [14]	2007+2012	70.0	0.5
Faster R-CNN VGG-16[28]	2007+2012	73.2	7
Faster R-CNN ZF [28]	2007+2012	62.1	18
YOLO VGG-16	2007+2012	66.4	21

Table 1: Real-Time Systems on PASCAL VOC 2007. Comparing the performance and speed of fast detectors. Fast YOLO is the fastest detector on record for PASCAL VOC detection and is still twice as accurate as any other real-time detector. YOLO is 10 mAP more accurate than the fast version while still well above real-time in speed.

Big Picture

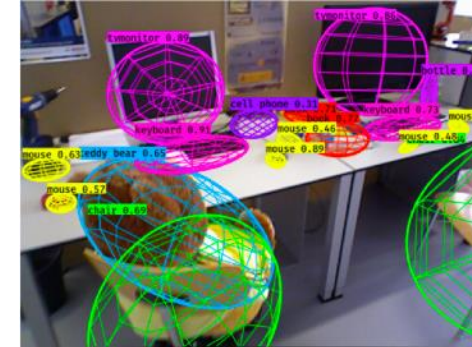


“Map”: Environment Representations

- **Sparse:**
 - Landmark-based
 - No explicit representation (pose graph)
 - Geometric primitives
- **Dense:**
 - Point clouds
 - 2D/3D occupancy grids
 - 3D meshes
 -



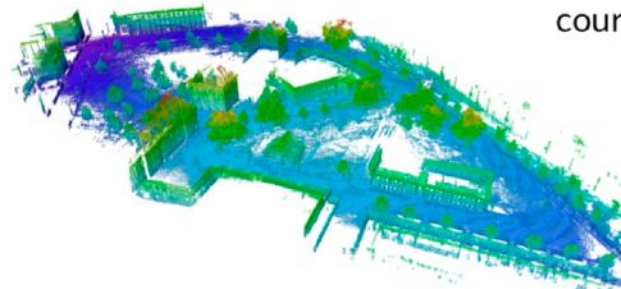
courtesy of Ranganathan et al.



courtesy of Nicholson et al.

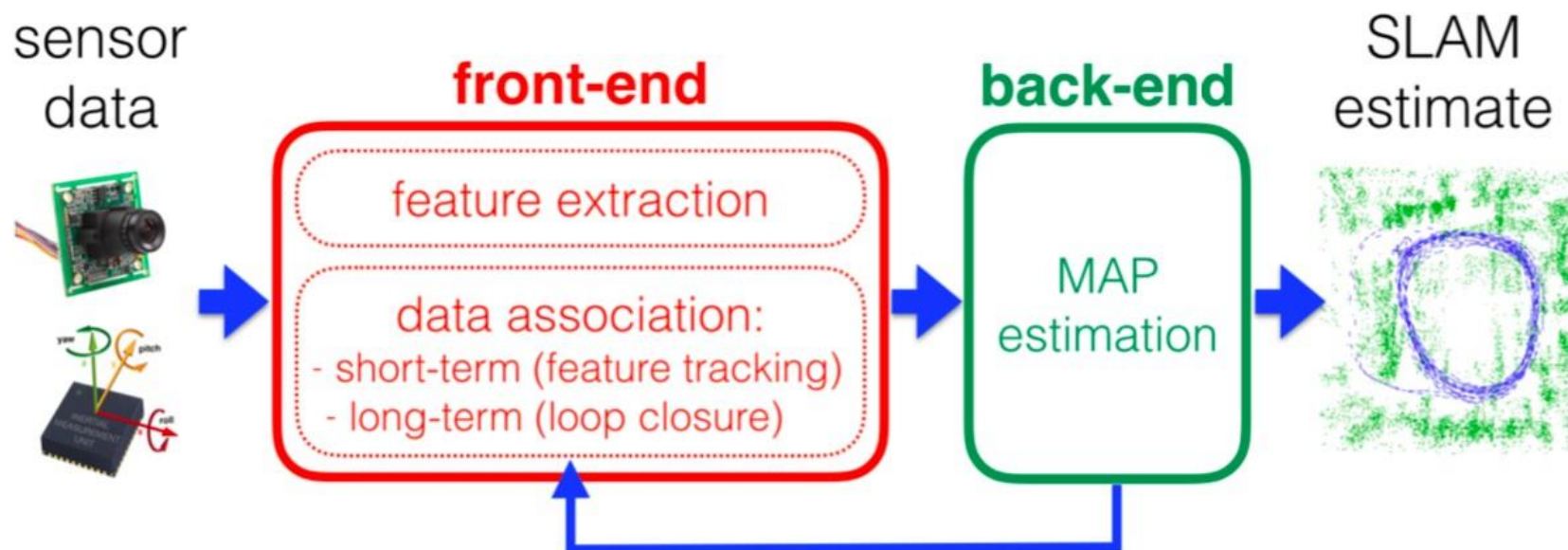


courtesy of Sameer Agarwal



courtesy of octomap

Some terminology



MAP is maximum *a posteriori* estimation
(MLE if no prior is available [“uninformative” prior])

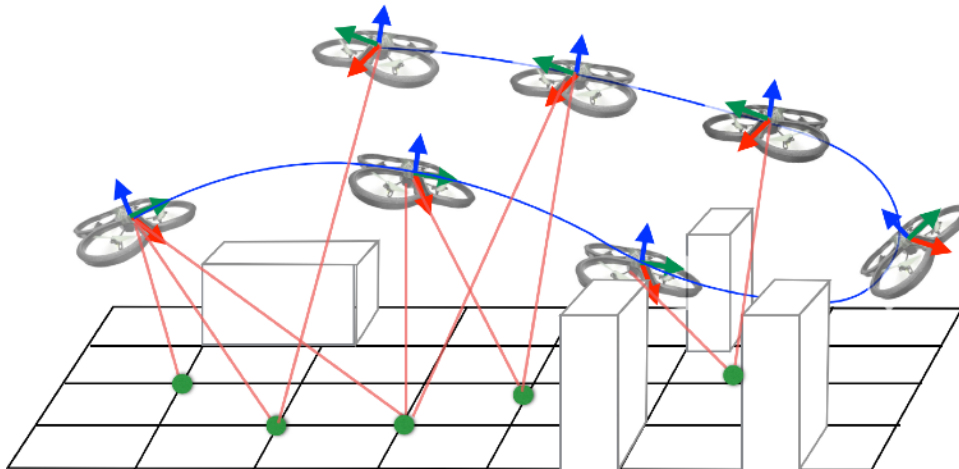
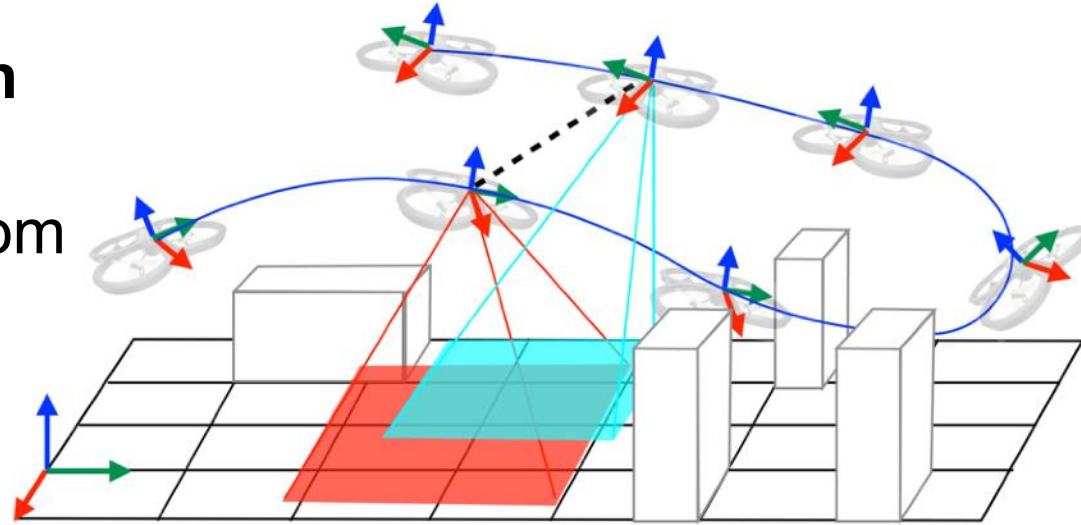
courtesy of Cadena et al.

Simultaneous Localization and Mapping

Pose Graph Optimization

(a.k.a. pose SLAM):

Estimate only trajectory from sensor data



Landmark-based SLAM:
Estimate trajectory of robot
and position of external
landmarks from sensor
data

SLAM and Factor Graph Formulation

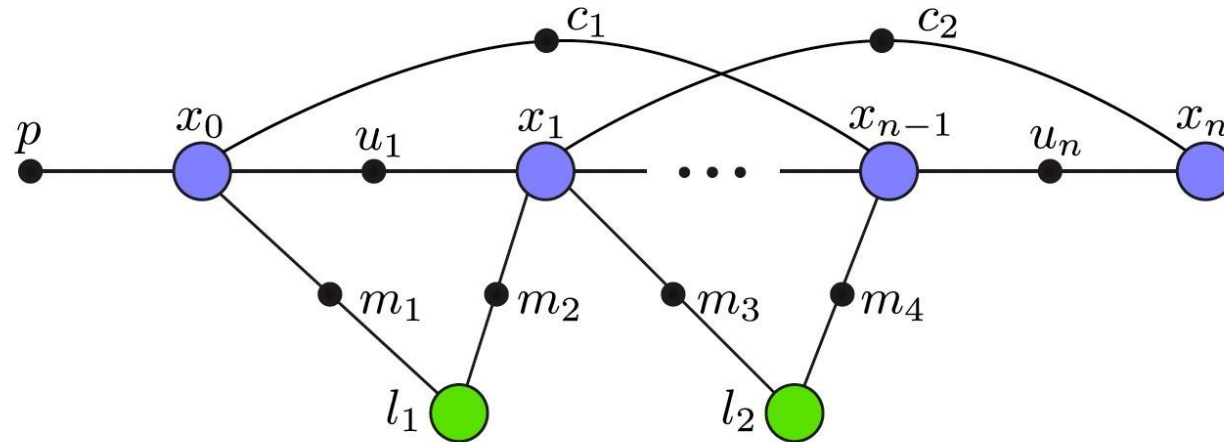
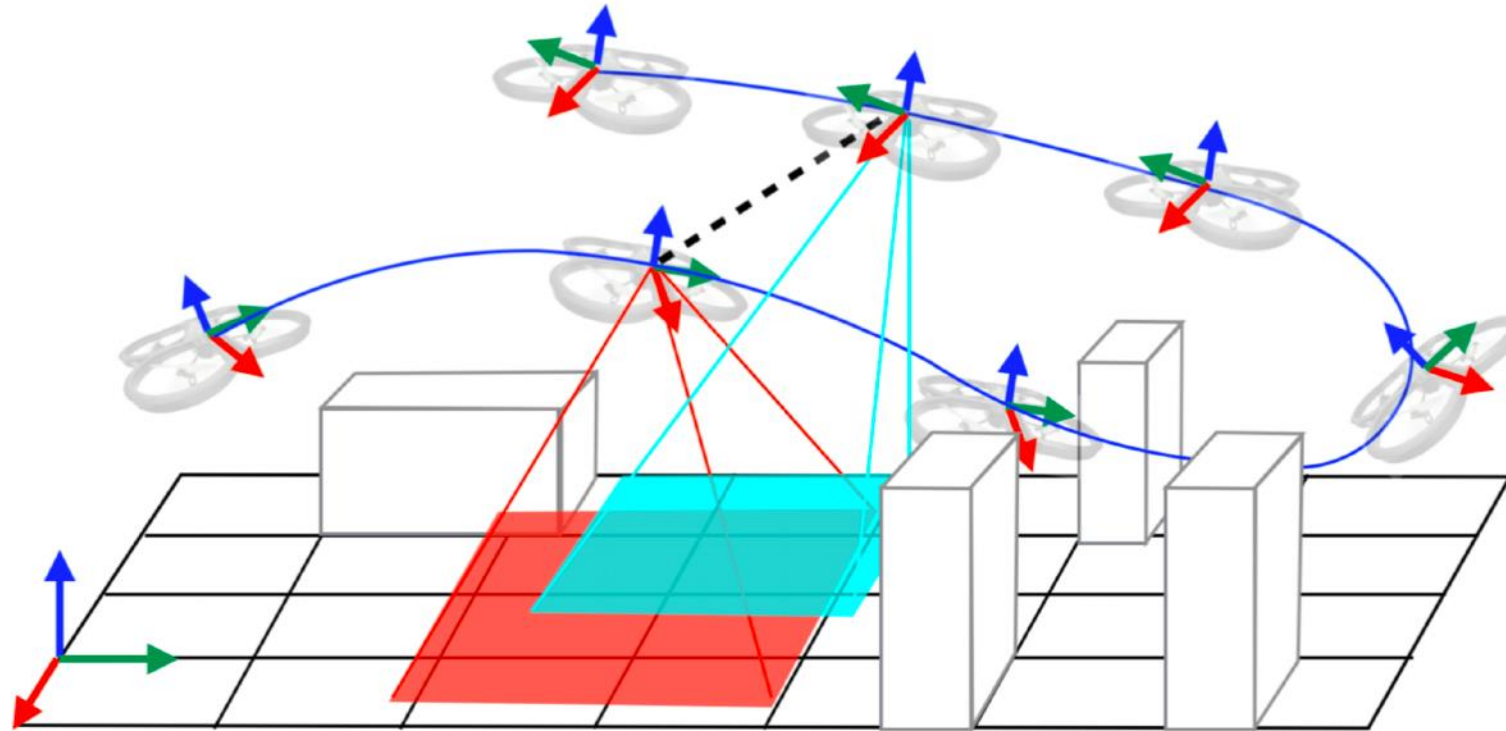


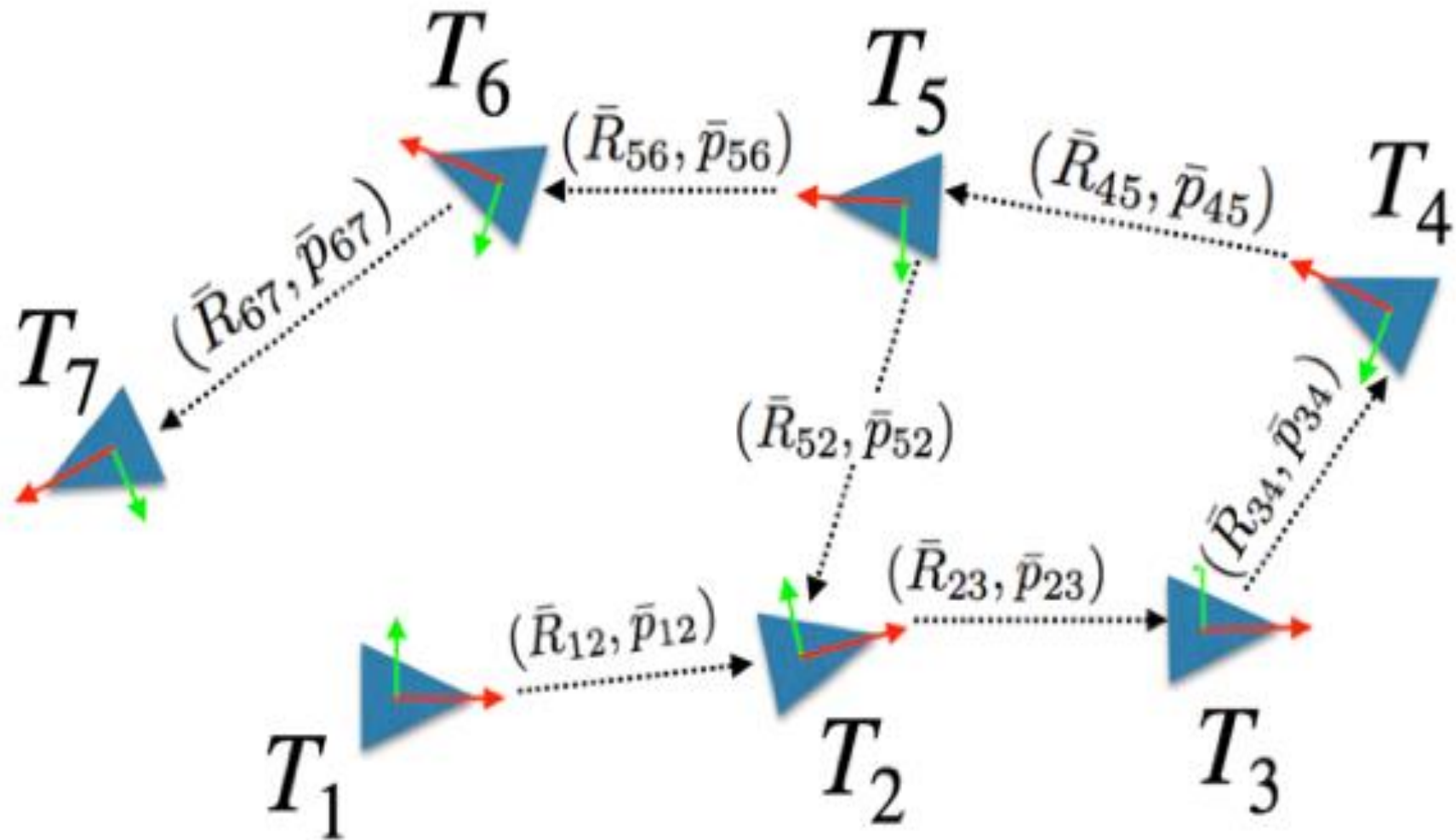
Fig. 2: Factor graph (Kschischang et al., 2001) formulation of the SLAM problem, where variable nodes are shown as large circles, and factor nodes (measurements) as small solid circles. The factors shown are odometry measurements u , a prior p , loop closing constraints c and landmark measurements m . Special cases include the pose-graph formulation (without l and m) and landmark-based SLAM (without c). Note that the factor graph can represent any cost function, involving one, two or more variables (e.g. calibration).

Pose Graph Optimization



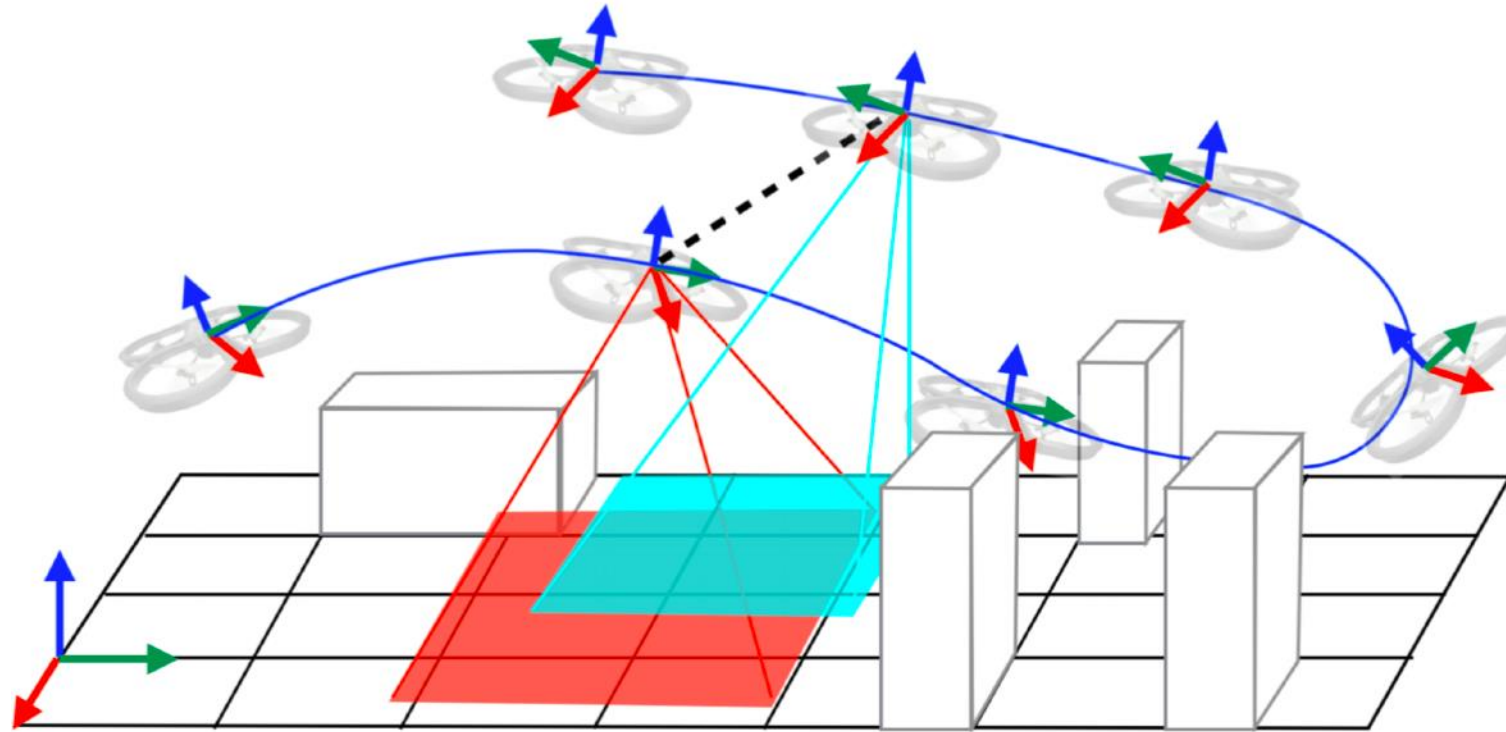
- **Measurements:** odometry + loop closures (i.e., relative pose measurements between non-consecutive poses obtained via place recognition & 2-view geometry, or similar)
- **Variables:** robot poses

Graphical representation of pose graph optimization



Pose graph example

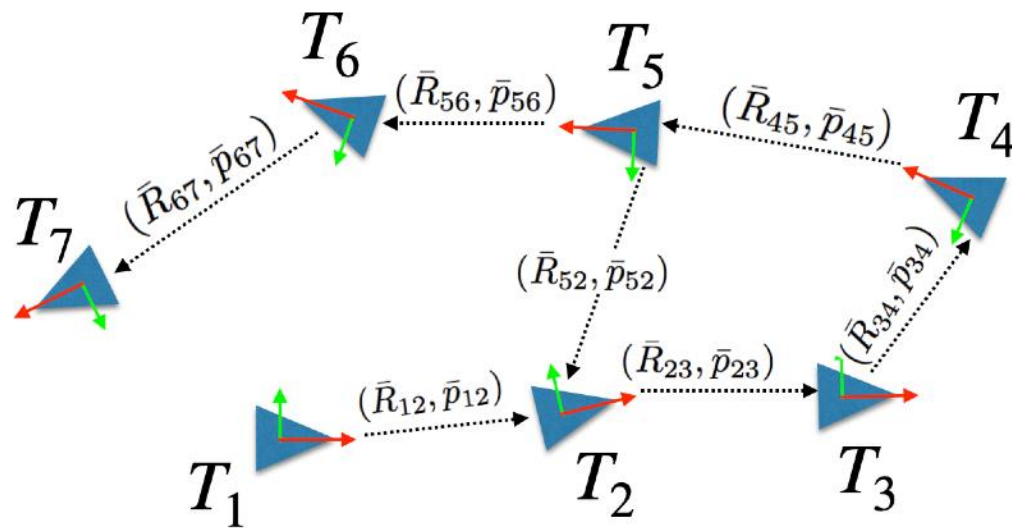
Pose Graph Optimization Recap



- **Measurements:** odometry + loop closures (relative poses)
- **Variables:** robot poses

Pose Graph Optimization

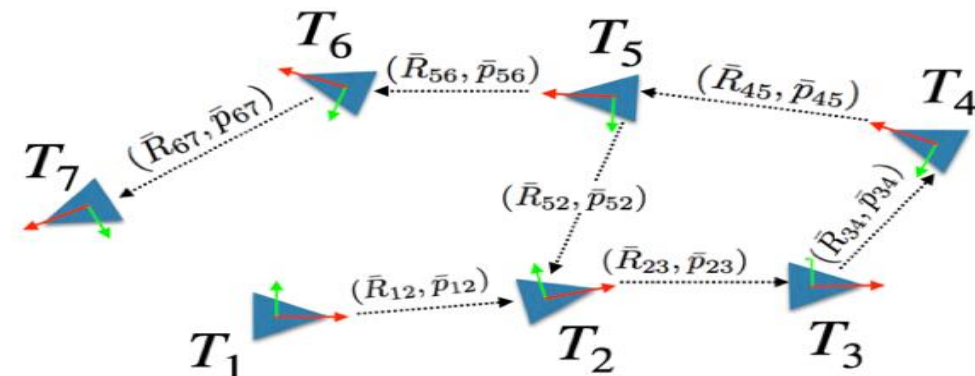
pose
graph



pose graph
optimization

$$\min_{\mathbf{T}_t, t=1, \dots, n} \sum_{(i,j) \in \mathcal{E}} \|(\mathbf{T}_i^{-1} \mathbf{T}_j) \ominus \bar{\mathbf{T}}_j^i\|_{\Sigma_{ij}}^2$$

Pose Graph Optimization: Sparsity



Jacobian **J**

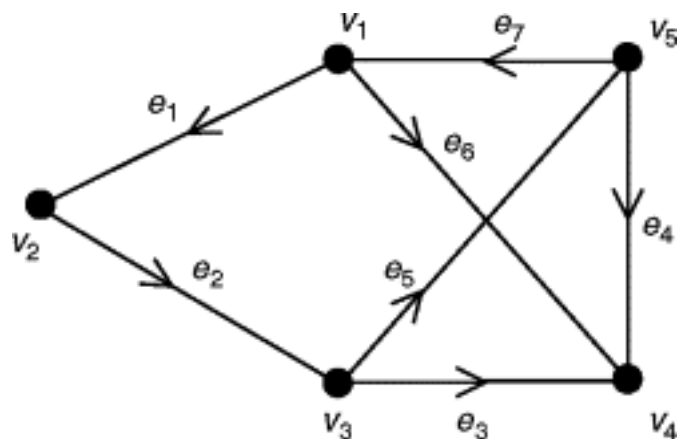
	T_1	T_2	T_3	T_4	T_5	T_6	T_7
$\bar{T}_{12}$	■	■					
$\bar{T}_{23}$		■	■				
$\bar{T}_{34}$			■	■			
$\bar{T}_{45}$				■	■		
$\bar{T}_{56}$					■	■	
$\bar{T}_{67}$						■	■
$\bar{T}_{52}$		■			■		

Hessian **J^T J**

	T_1	T_2	T_3	T_4	T_5	T_6	T_7
T_1	■	■					
T_2	■	■	■		■		
T_3		■	■	■			
T_4			■	■	■		
T_5		■		■	■	■	
T_6					■	■	■
T_7						■	■

a.k.a.
**Information
 Matrix** of
 the estimate

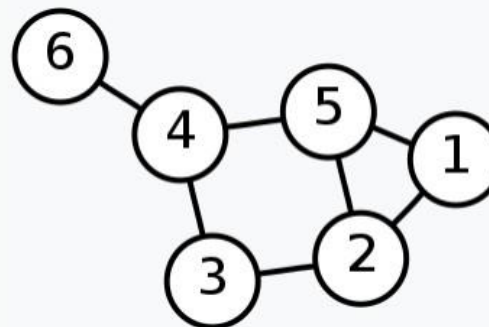
Graphical representation of pose graph optimization



Incidence Matrix

$$\begin{array}{c}
 v_1 \\
 v_2 \\
 v_3 \\
 v_4 \\
 v_5
 \end{array}
 \begin{bmatrix}
 \theta_1 & \theta_2 & \theta_3 & \theta_4 & \theta_5 & \theta_6 & \theta_7 \\
 1 & 0 & 0 & 0 & 0 & 1 & -1 \\
 -1 & 1 & 0 & 0 & 0 & 0 & 0 \\
 0 & -1 & 1 & 0 & 1 & 0 & 0 \\
 0 & 0 & -1 & -1 & 0 & -1 & 0 \\
 0 & 0 & 0 & 1 & -1 & 0 & 1
 \end{bmatrix}$$

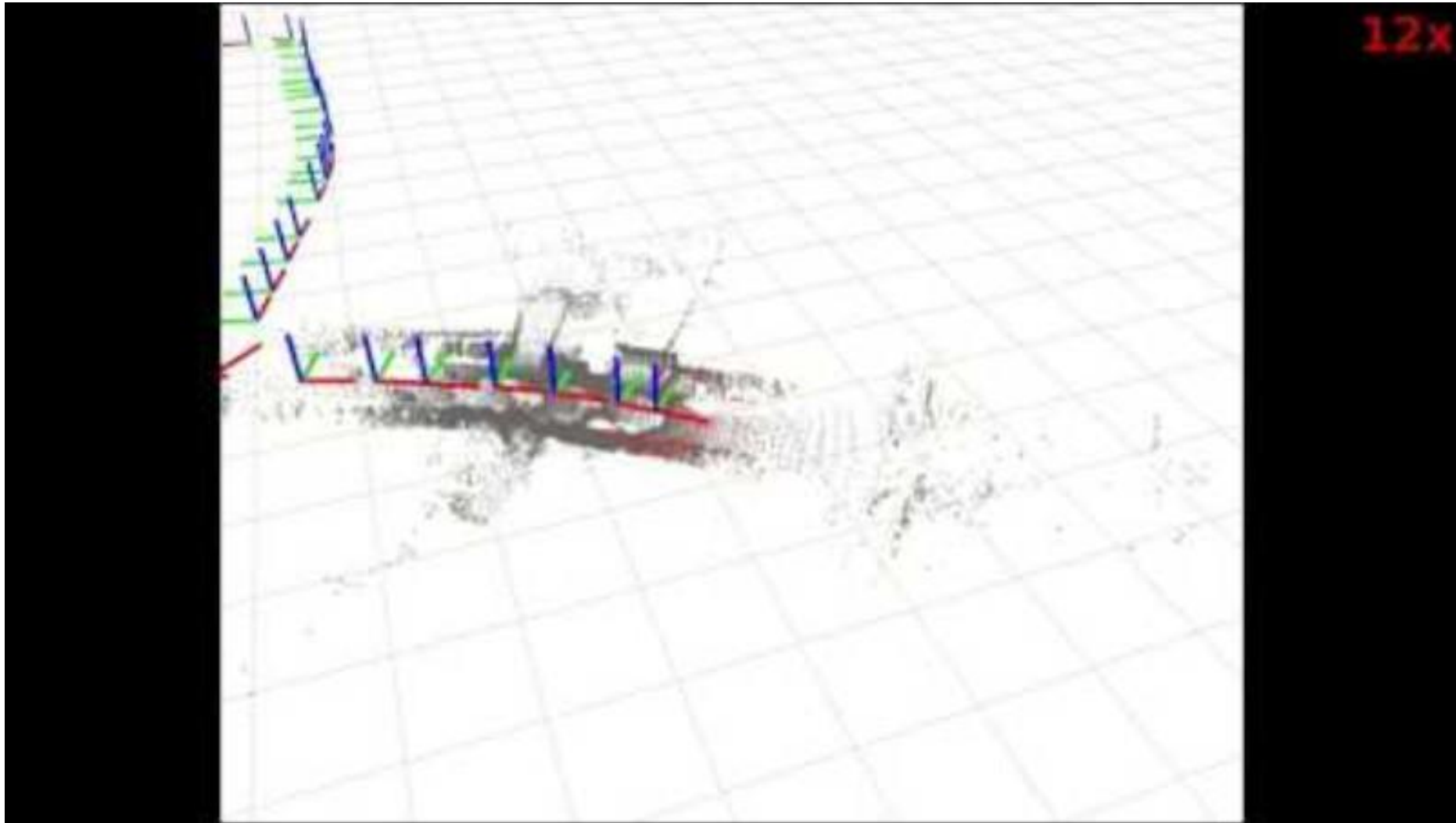
Labelled graph



Laplacian matrix

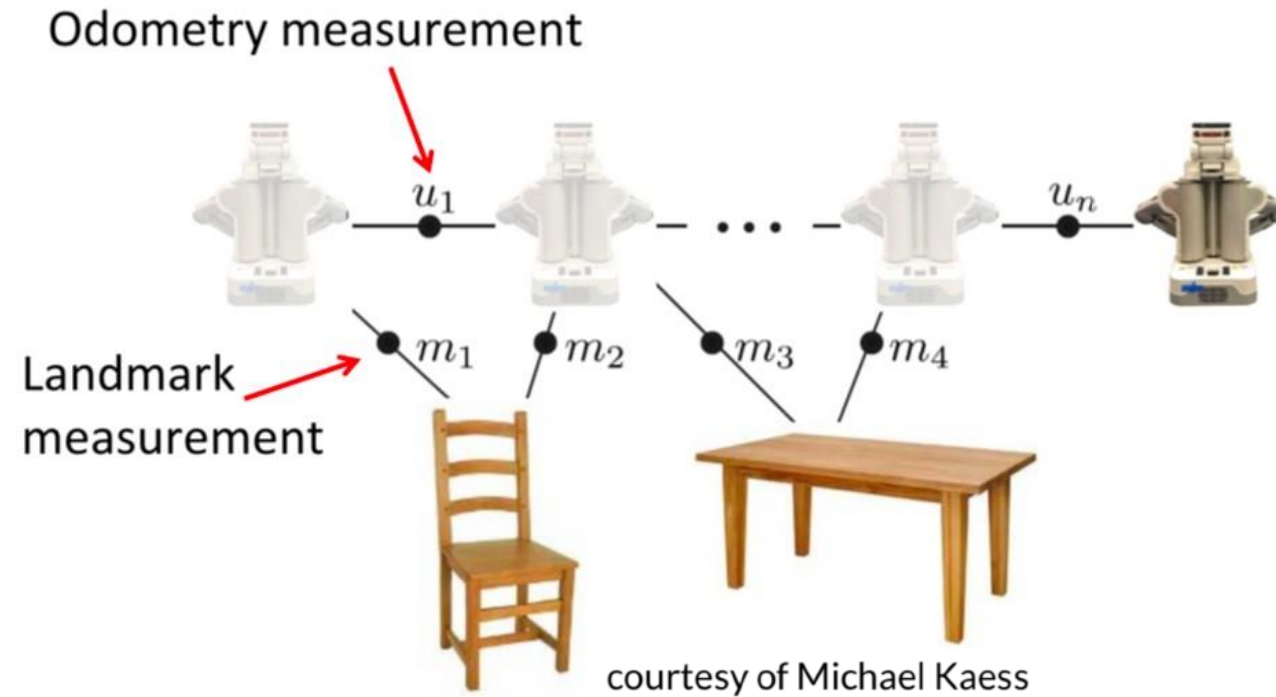
$$\begin{pmatrix}
 2 & -1 & 0 & 0 & -1 & 0 \\
 -1 & 3 & -1 & 0 & -1 & 0 \\
 0 & -1 & 2 & -1 & 0 & 0 \\
 0 & 0 & -1 & 3 & -1 & -1 \\
 -1 & -1 & 0 & -1 & 3 & 0 \\
 0 & 0 & 0 & -1 & 0 & 1
 \end{pmatrix}$$

Pose Graph Optimization: Example



https://www.youtube.com/watch?v=KYvOqUB_odg

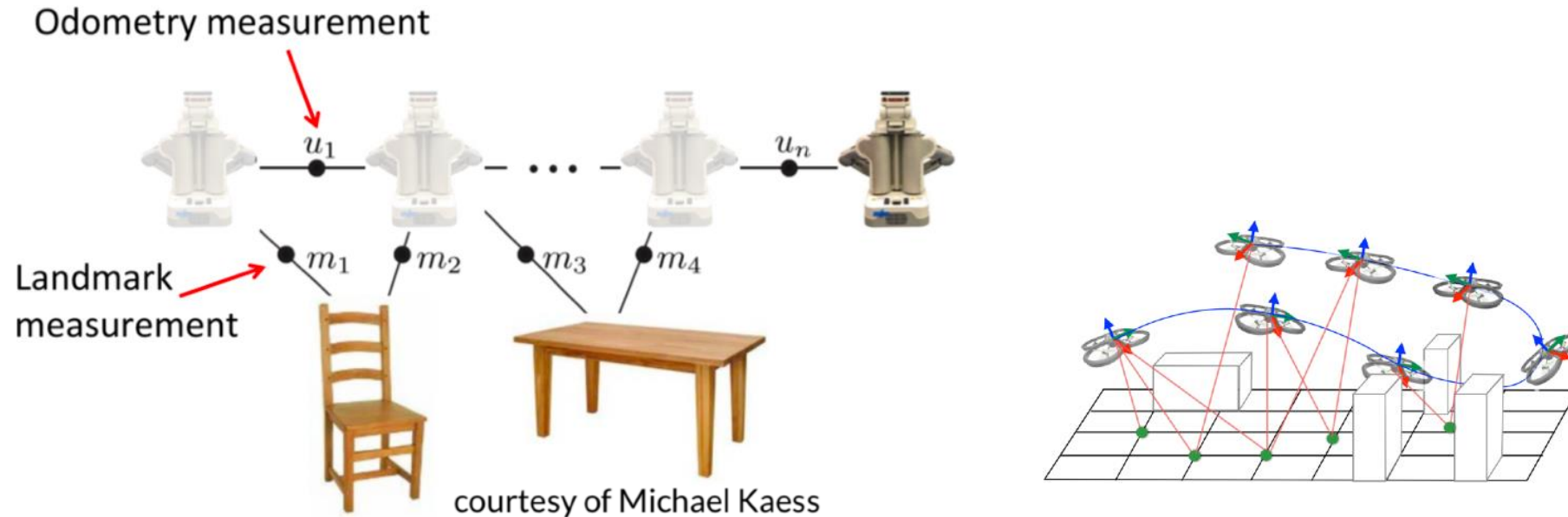
Graphical representation of landmark-based SLAM



- ▶ Each variable (robot pose, landmark position/pose) is a node in the graph
- ▶ Each (usually) pairwise measurement denotes an edge between the corresponding two variables (nodes)

Landmark-based SLAM

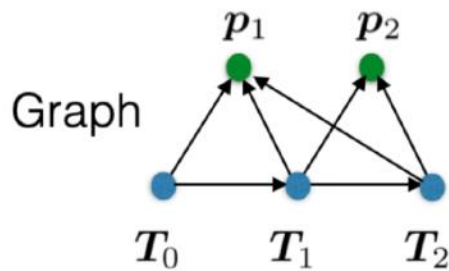
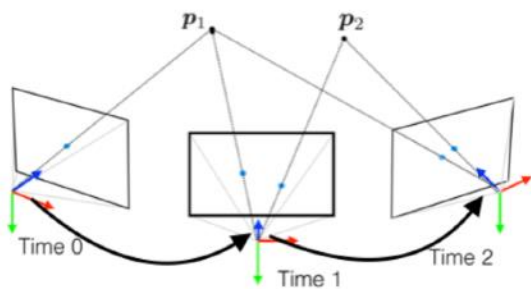
- ▶ Sequence of robot (camera) poses $\mathbf{T}_1, \mathbf{T}_2, \dots, \mathbf{T}_t \in \text{SE}(d)$
- ▶ Robot measures the relative pose between $\mathbf{T}_i$ and $\mathbf{T}_{i+1}$ (odometry)
- ▶ Robot measures the environment (e.g., point landmarks $\mathbf{p}_i \in \mathbb{R}^d$)



- **Measurements:** odometry + measurements of (projection, range, position, or others) of external landmarks
- **Variables:** robot poses and landmark positions

Landmark-based SLAM: Sparsity

$$\min_{\substack{\mathbf{T}_t, t=1, \dots, n \\ \mathbf{l}_k, k=1, \dots, K}} \sum_{t=1, \dots, n-1} \|(\mathbf{T}_t^{-1} \mathbf{T}_{t+1}) \boxminus \bar{\mathbf{T}}_{t+1}^t\|_{\Sigma_o}^2 + \sum_{k=1, \dots, K} \sum_{t \in S_k} \|\bar{\mathbf{y}}_{k,t} - h_i(\mathbf{T}_t, \mathbf{l}_k)\|_{\Sigma_l}^2$$



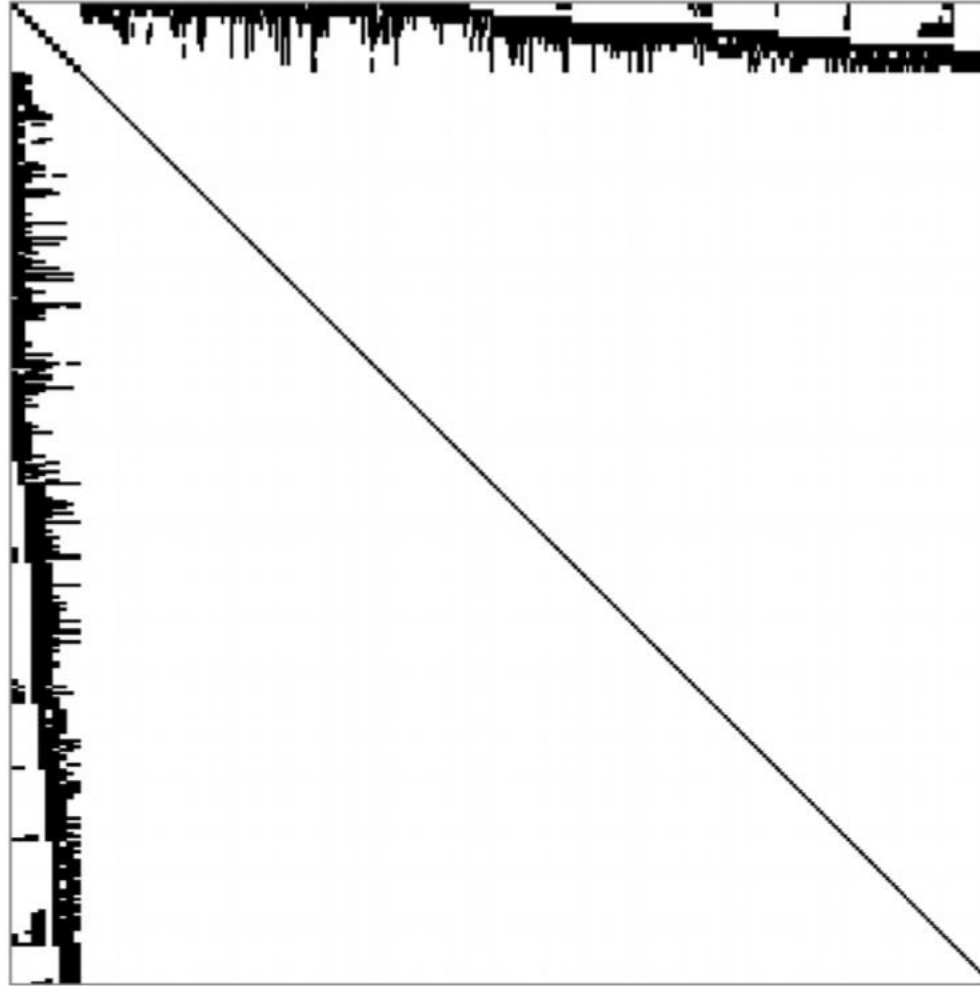
Jacobian $\mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0	Blue	Blue			
T_1		Blue	Blue		
T_2			Blue	Blue	
p_1				Blue	
p_2					Blue

Hessian $\mathbf{J}^T \mathbf{J}$

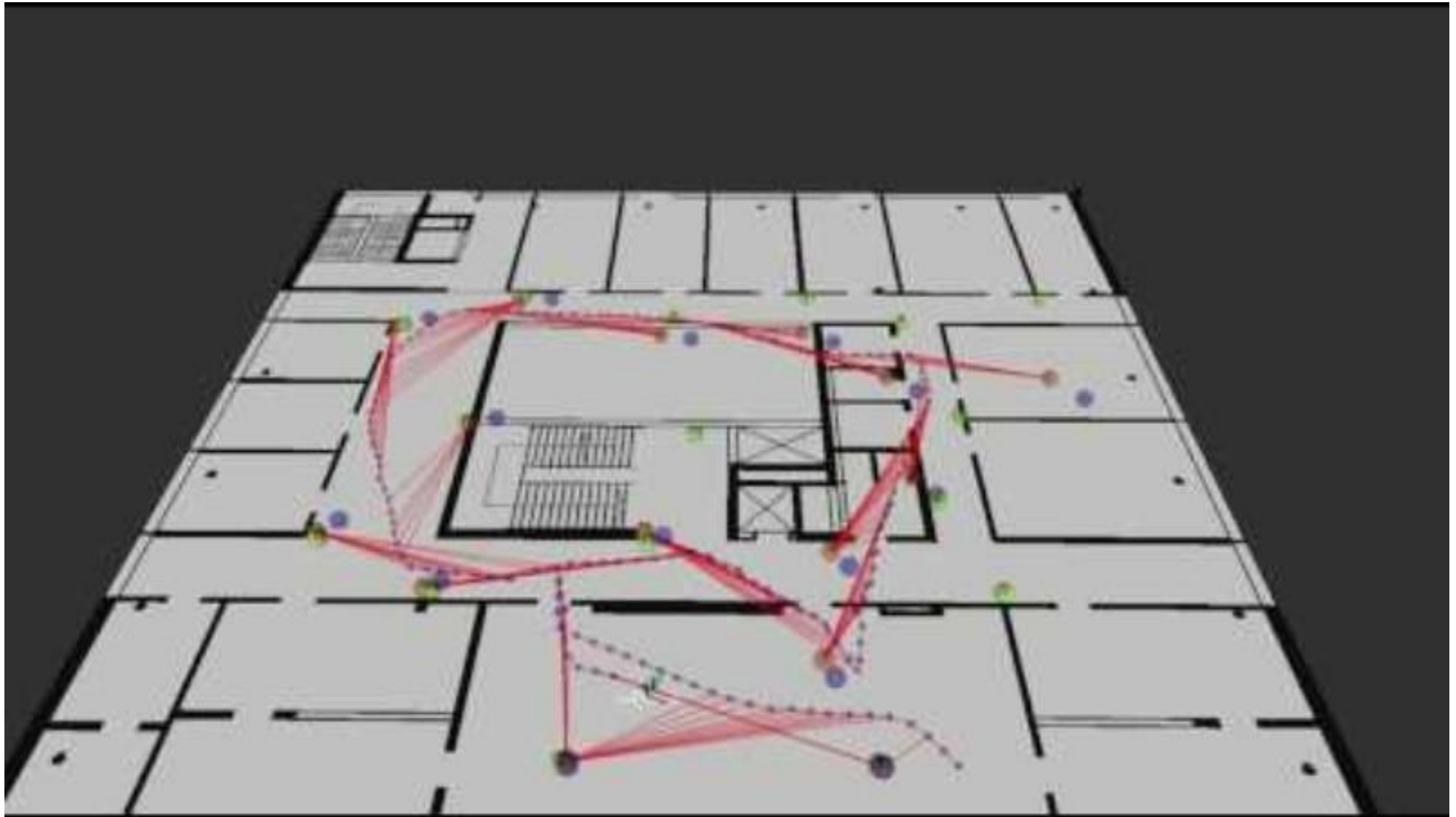
	T_0	T_1	T_2	p_1	p_2
T_0	Blue	Blue		Light Blue	
T_1		Blue	Blue	Light Blue	Light Blue
T_2			Blue	Light Blue	Light Blue
p_1				Blue	
p_2					Blue

Example of Hessian (sparsity)



Credit: *Lourakis and Argyros*

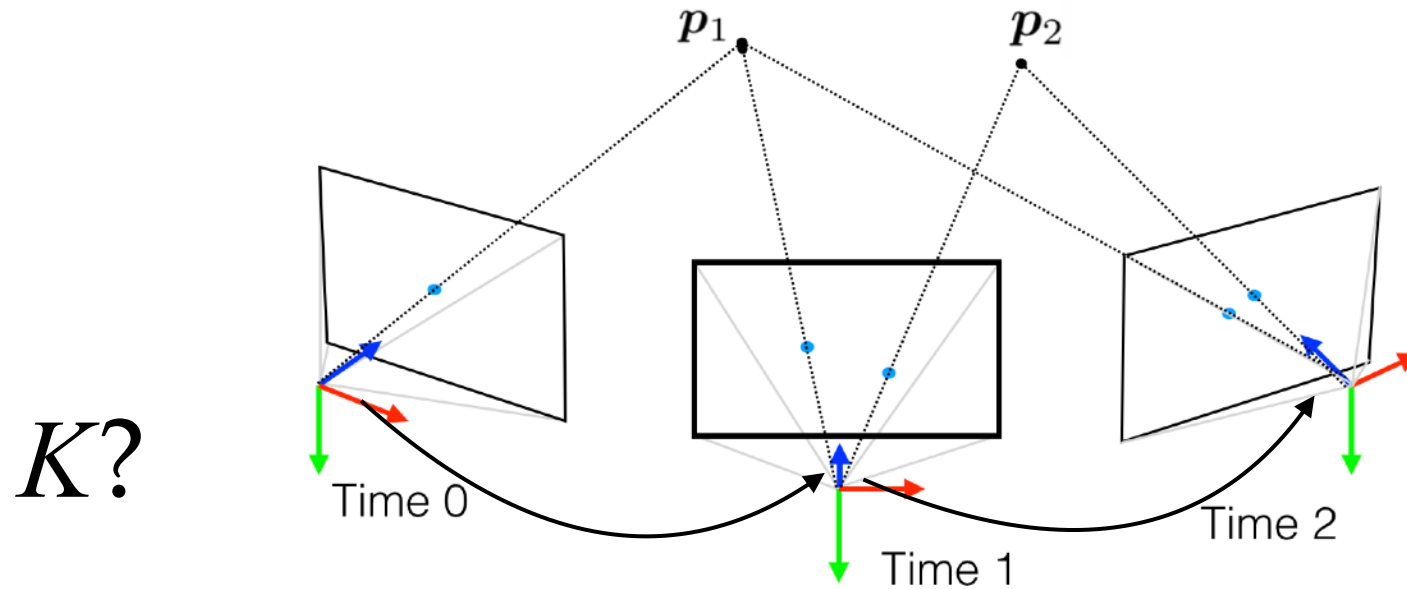
Landmark-based SLAM: Example



<http://www.cs.cmu.edu/~mroberts/363/06slam/06slam.html>

Other SLAM Problems

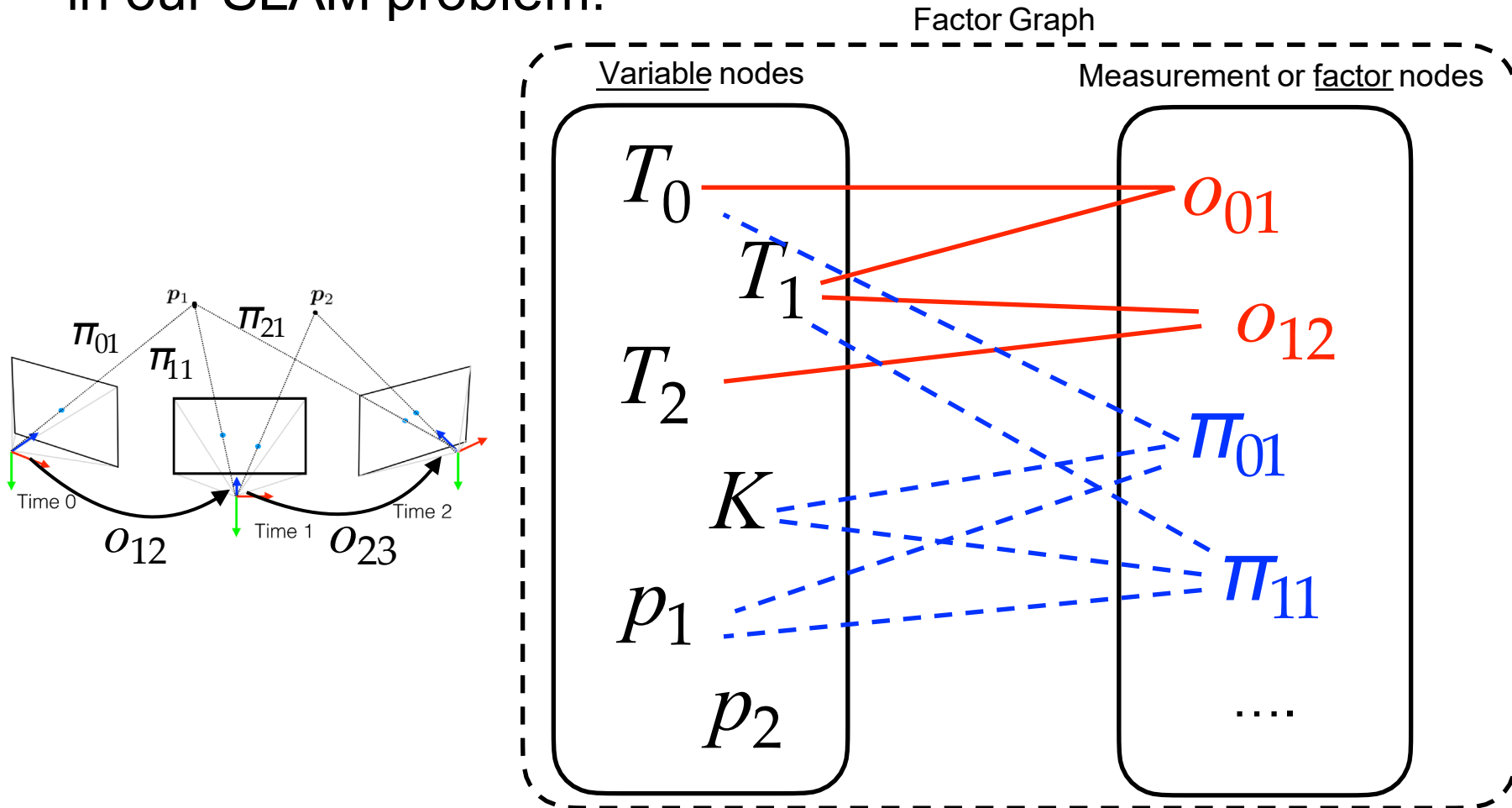
- Consider a visual-SLAM problem where we also want to estimate the camera calibration:



Problem: the projective measurements depend on (i) a pose, (ii) a 3D point, and (iii) the unknown calibration. We can no longer use a standard graph representation where measurements are (pairwise) edges

A General Model: Factor Graphs

- Bipartite graph describing measurements and variables in our SLAM problem:



Factor Graph: Example

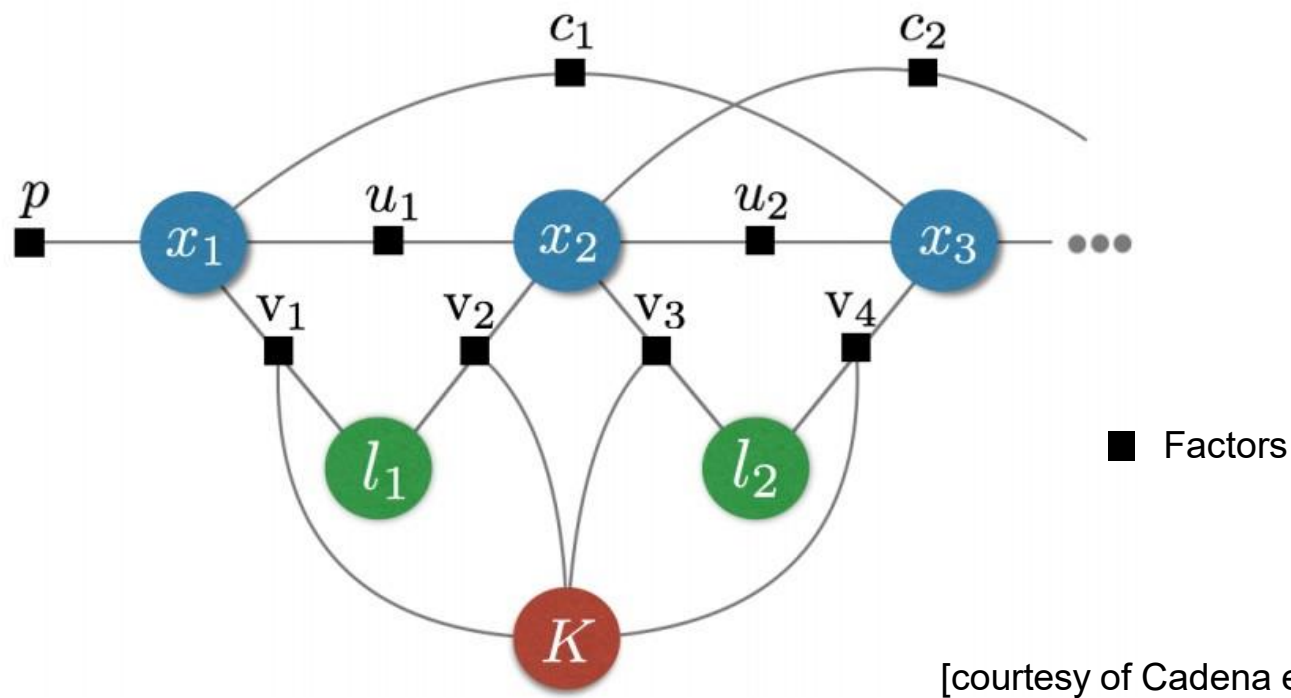
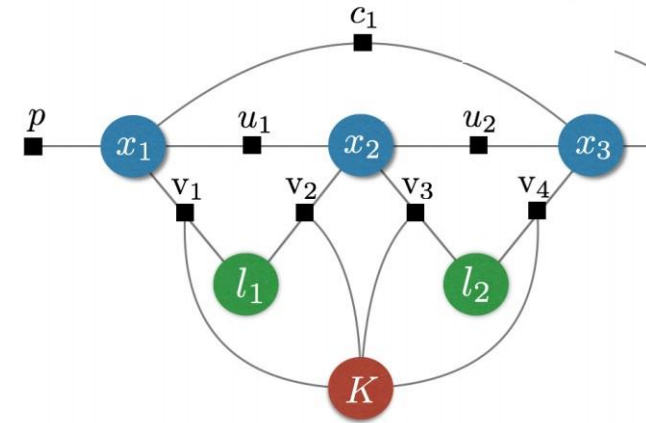


Fig. 3: **SLAM as a factor graph**: Blue circles denote robot poses at consecutive time steps ($x_1, x_2, \dots$), green circles denote landmark positions ($l_1, l_2, \dots$), red circle denotes the variable associated with the intrinsic calibration parameters (K). Factors are shown as black squares: the label “u” marks factors corresponding to odometry constraints, “v” marks factors corresponding to camera observations, “c” denotes loop closures, and “p” denotes prior factors.

Factor Graph: Sparsity

- Sparsity is dictated by topology of the factor graph:



Jacobian $\mathbf{J}$

	x_1	x_2	x_3	l_1	l_2	K
p	■					
u_1	■	■				
u_2		■	■			
v_1	■			■		■
v_2		■		■		■
v_3		■			■	■
v_4			■		■	■
c_1	■		■			

Hessian $\mathbf{J}^T \Sigma^{-1} \mathbf{J}$

	x_1	x_2	x_3	l_1	l_2	K
x_1	■	■	■	■		■
x_2	■	■	■	■	■	■
x_3	■	■	■		■	■
l_1	■	■		■		■
l_2		■	■		■	■
K	■	■	■	■	■	■

a.k.a.
Information
Matrix of
the estimate

► Normal equations: $(\mathbf{J}^T \Sigma^{-1} \mathbf{J}) \mathbf{d} = -\mathbf{J}^T \Sigma^{-1} \mathbf{r}$

What if we only care about subset of variables?

► Normal equations: $(\mathbf{J}^\top \Sigma^{-1} \mathbf{J}) \mathbf{d} = -\mathbf{J}^\top \Sigma^{-1} \mathbf{r}$

• What if we only want to compute a subset of variables?

► $\mathbf{J} = [\mathbf{J}_p \quad \mathbf{J}_l]$, i.e., partial derivatives w.r.t. poses and w.r.t. landmarks

► Information matrix (LHS) blocks

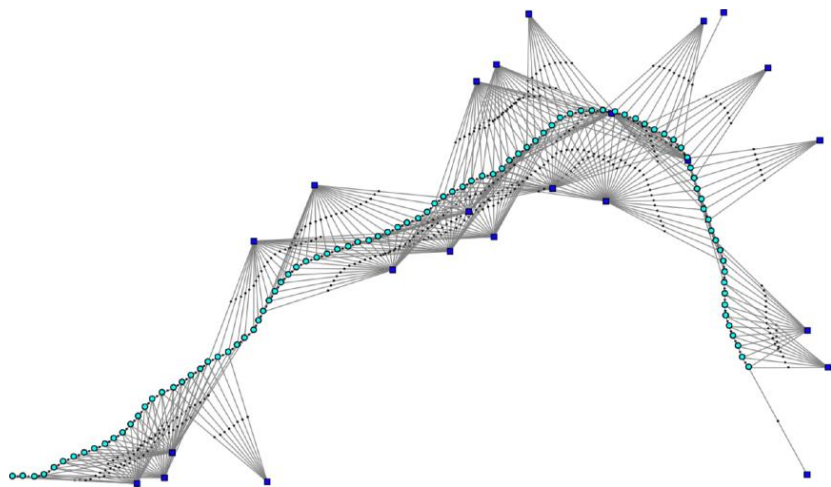
Block structure
in the Information
Matrix

$$\mathbf{J}^\top \Sigma^{-1} \mathbf{J} = \begin{bmatrix} \mathbf{J}_p^\top \Sigma^{-1} \mathbf{J}_p & \mathbf{J}_p^\top \Sigma^{-1} \mathbf{J}_l \\ \mathbf{J}_l^\top \Sigma^{-1} \mathbf{J}_p & \mathbf{J}_l^\top \Sigma^{-1} \mathbf{J}_l \end{bmatrix} =: \begin{bmatrix} \mathbf{H}_{pp} & \mathbf{H}_{pl} \\ \mathbf{H}_{pl}^\top & \mathbf{H}_{ll} \end{bmatrix}$$



$$\begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\top & \mathbf{C} \end{bmatrix} \begin{bmatrix} \mathbf{x} \\ \mathbf{y} \end{bmatrix} = \begin{bmatrix} \mathbf{w} \\ \mathbf{z} \end{bmatrix}$$

Schur Complement & Marginalization



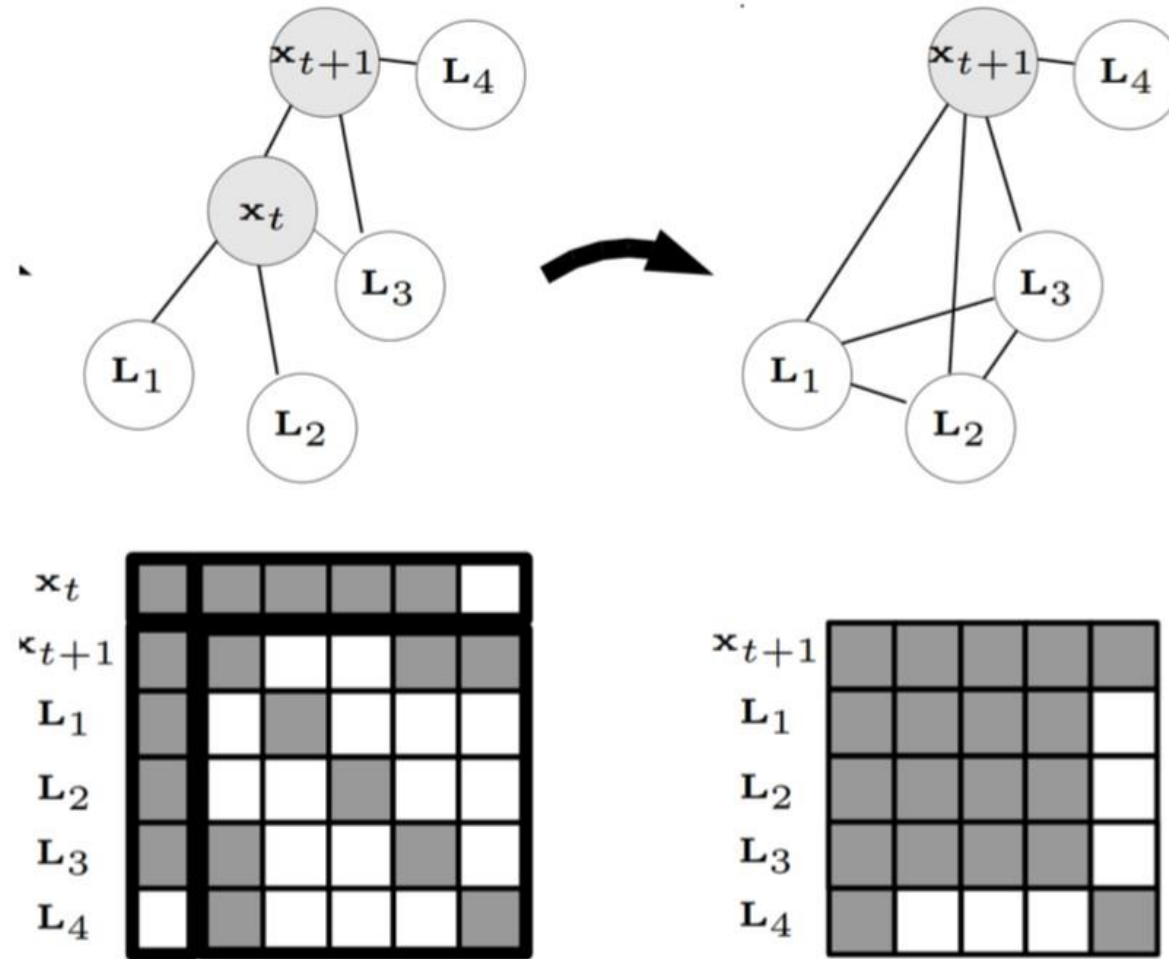
- ▶ Many times we may wish to forget/eliminate unimportant variables (to focus resources on what matters to us, reduce size of linear system, save memory, etc)
- ▶ How to eliminate (forget) some variables “without” loss of information?
- ✗ Naïvely discarding variables and their measurements → loss of information
- ✓ Proper way: *Marginalize* them out

Schur Complement & Marginalization

- ▶ What does marginalization/Schur complement do to the sparsity pattern of information matrix?
- ▶ Eliminating (marginalizing out) a variable creates non-zero off-diagonals (called *fill-in*) in the information matrix between all of its “neighbours” (i.e., those variables that had a non-zero off-diagonal with the eliminated variable in the information matrix)
- ▶ In graph terms, elimination creates a *clique* between the neighbours of the eliminated node

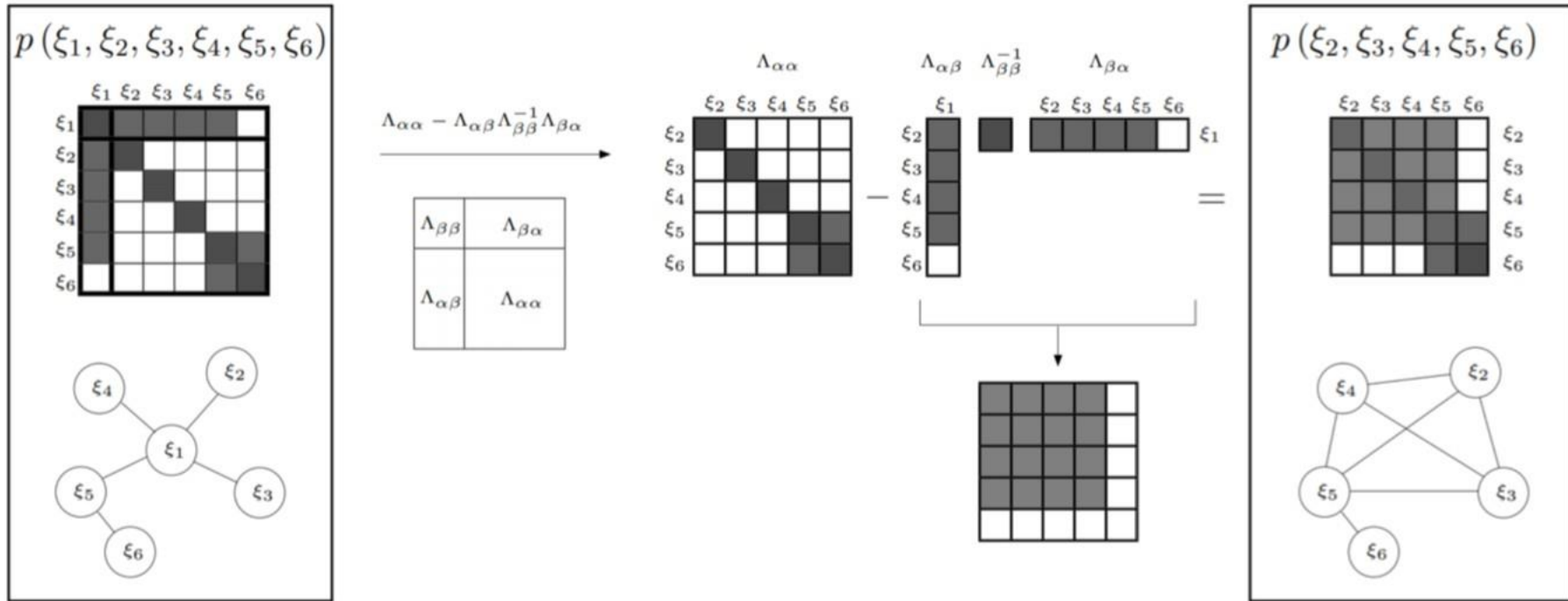
⇒ Loss of sparsity!

Marginalization: Example 1



Marginalization: Example 2

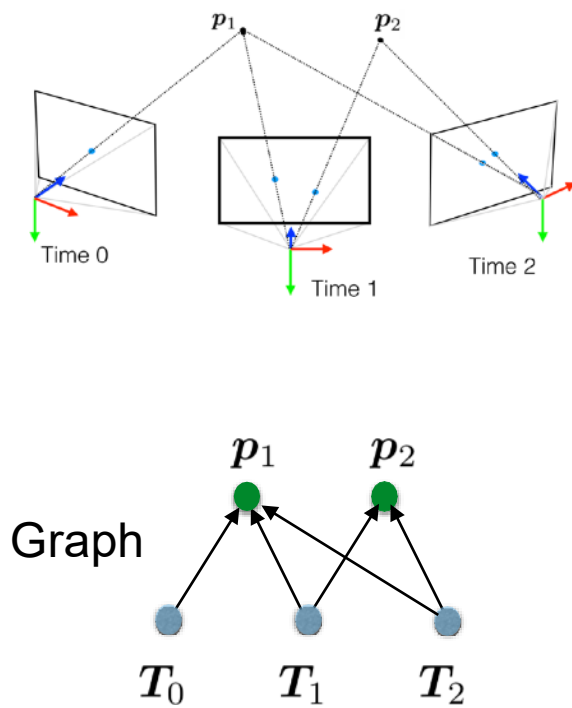
Marginalize ξ_1



Credit: Walter et al.

Backup

Windowed Bundle Adjustment



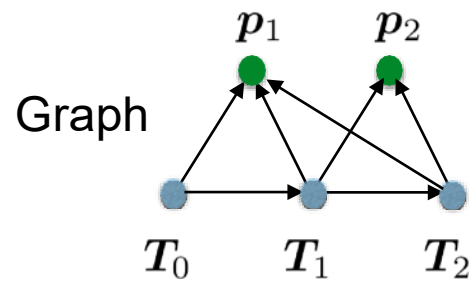
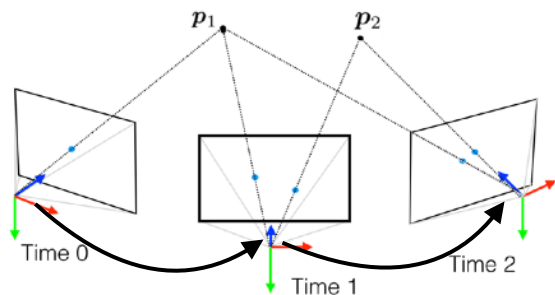
Jacobian $\mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0					
T_1					
T_2					
p_1					
p_2					

Hessian $\mathbf{J}^T \mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0					
T_1					
T_2					
p_1					
p_2					

Windowed Bundle Adjustment



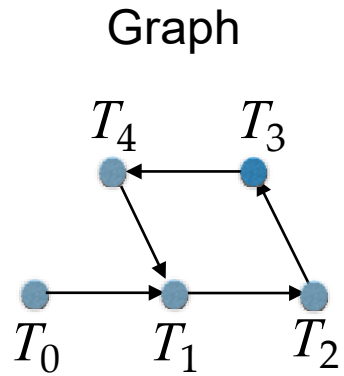
Jacobian $\mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0	Blue	Blue			
T_1		Blue	Blue		
T_2			Blue	Blue	
p_1		Blue	Blue	Blue	
p_2		Blue	Blue	Blue	Blue

Hessian $\mathbf{J}^T \mathbf{J}$

	T_0	T_1	T_2	p_1	p_2
T_0	Blue	Blue		Light Blue	
T_1	Blue	Blue	Blue	Light Blue	Light Blue
T_2		Blue	Blue	Light Blue	Light Blue
p_1	Light Blue	Light Blue	Light Blue	Blue	
p_2		Light Blue	Light Blue		Blue

Windowed Bundle Adjustment



Transpose
Incidence Matrix

	T_0	T_1	T_2	T_3	T_4
$\bar{T}_{01}$	-1	1			
$\bar{T}_{12}$		-1	1		
$\bar{T}_{23}$			-1	1	
$\bar{T}_{34}$				-1	1
$\bar{T}_{41}$		1			-1

Laplacian
Matrix

	T_0	T_1	T_2	T_3	T_4
T_0	1	-1			
T_1	-1	3	-1		-1
T_2		-1	2	-1	
T_3			-1	2	-1
T_4		-1		-1	2

Clique Set (from Wikipedia)

A **clique**, C , in an **undirected graph** $G = (V, E)$ is a subset of the **vertices**, $C \subseteq V$, such that every two distinct vertices are adjacent.

A **maximal clique** is a clique that cannot be extended by including one more adjacent vertex, that is, a clique which does not exist exclusively within the vertex set of a larger clique.

A **maximum clique** of a graph, G , is a clique, such that there is no clique with more vertices.